CSC322 – OPERATING SYSTEMS
LAB NO. 05
DETAILED LECTURE & PRACTICAL NOTES

Managing Processes, and Writing, Compiling & Executing C++ on Linux

Prepared directly from the provided CSC322 Lab Manual, with detailed command descriptions and simple classroom explanations.

1. Lab Objective

This lab is designed to demonstrate basic task/process management activities and introduce writing C++ programs in Linux.

2. Activity Outcomes

- Manage processes in Linux.
- Compile and execute C++ programs using the command line in Linux.

3. Instructor Note / Pre-Lab

The manual recommends reading Chapter 10 from “The Linux Command Line” by William E. Shotts, Jr.

4. What is a Process?

A process is an instance of a computer program that is being executed. A program is a passive collection of instructions; a process is the actual execution of those instructions.

Very easy example:

```
hello.cpp  → source/program
./hello     → running program = process
```

The same program can have several processes if it is opened/run multiple times.

The Linux kernel keeps information about processes, including their PID, memory and execution status. The manual notes that init has PID 1.

5. PID – Process ID

PID means Process ID. It is a unique number used by Linux to identify a running process.

```
ps
ps -p 1
```

In ps output, look for the PID column. You can then use that PID with commands such as ps -p PID or renice.

6. Static Process Monitoring – ps

The ps command gives a snapshot (static view) of processes.

Command	Purpose	Example	Easy Explanation
ps	Processes in current shell	ps	Shows processes related to the current terminal/shell.
ps -A	Every active process	ps -A	Shows processes running throughout the system.
ps -e	Every active process	ps -e	Alternative form commonly used for all processes.
ps -x	Processes owned by current user	ps -x	Shows your processes, including ones without a controlling terminal.
ps -F	Full-format listing	ps -F	Displays more detailed process information.
ps -u USER	Processes for a user	ps -u ubuntu	Shows processes associated with user ubuntu.
ps -p PID	Specific process	ps -p 1	Shows information for the process whose PID is 1.
ps -r	Running processes	ps -r	Selects processes currently in running state.
ps -L	Threads	ps -L	Shows threads associated with processes.
ps -G GROUP	Processes by group	ps -G users	Selects processes associated with a group.

7. Understanding Common ps Output

Typical columns can include PID, TTY, STAT, TIME and CMD. PID identifies the process; STAT describes its state; CMD indicates the command/program.

```
ps -F
ps -p 1 -F
```

Teaching point: do not memorize every column. Students should first understand PID, process state, and command name.

8. Process States

Symbol	Meaning	Easy Explanation
R	Running	Currently running or ready to run.
S	Sleeping	Waiting for an event/resource.
D	Uninterruptible sleep	Usually waiting for I/O.
T	Stopped	Process has been stopped/suspended.
Z	Zombie	Process has finished but still has an entry awaiting parent handling.

l	Multithreaded	Process has multiple threads.
s	Session leader	Process is a session leader.
+	Foreground	Associated with the foreground process group.
<	High priority	High scheduling priority indication.
N	Low priority	Low scheduling priority indication.

9. top – Dynamic Process Monitoring

top displays a live/dynamic view of processes and system resource usage.

```
top
```

The display keeps updating. Use q to quit.

```
top -d 0.5
```

Here -d 0.5 means refresh approximately every 0.5 seconds.

```
top -u ubuntu
```

This displays the dynamic process view for the specified user.

Teaching example: ps is like taking a photograph of processes; top is like watching a live video.

10. pstree – Process Tree

```
pstree
```

pstree shows processes in a tree structure. It helps students visualize parent-child relationships.

```
Example idea:
init/systemd
 ├── process A
 │     └── child process
 └── process B
```

11. Foreground vs Background

Foreground process: uses/occupies the terminal for the command. Background process: runs without keeping the terminal blocked.

```
gedit &
```

The & tells the shell to start the command in the background.

```
jobs
```

jobs lists jobs controlled by the current shell.

```
fg
```

fg brings a background/suspended job to the foreground.

```
bg
```

bg resumes a stopped job in the background.

Keyboard controls:

- Ctrl+Z → suspend the foreground job.
- Ctrl+C → send an interrupt to the foreground command.

12. Exact Activity 1 – Process Monitoring

The manual asks students to perform the following tasks.

#	Task	Command/Action	Explanation
1	Display all processes in current shell	ps	Run ps and inspect PID/command.
2	Display every active process	ps -A	Use this when you want a system-wide process snapshot.
3	Full-format processes owned by you	ps -x -F	Shows more details for your user.
4	Full-format processes owned by ubuntu	ps -u ubuntu -F	Replace ubuntu with the required username if necessary.
5	Display process with ID 1	ps -p 1	Selects PID 1.
6	Dynamic view; refresh 0.5 sec	top -d 0.5	Press q to exit.
7	Dynamic view for user 999 or ubuntu	top -u ubuntu	Use the actual available username when appropriate.
8	Start gedit in background, then foreground	gedit & then fg	Use jobs/fg if multiple jobs exist.
9	Suspend gedit	Ctrl+Z	Stops/suspends the foreground job.
10	Resume gedit	bg	Continues the suspended job in background.

13. Activity 1 – Classroom Demonstration

1. Open Terminal.
2. Run ps and ask students: What is the PID?
3. Run ps -A and explain that many system processes are present.
4. Run ps -F and explain that it gives more information.
5. Run ps -p 1 and discuss PID 1.
6. Run top -d 0.5. Explain the live display and press q.
7. Run pstree and point out parent-child relationships.
8. Run gedit &. Explain why the prompt returns immediately.
9. Run jobs.
10. Use fg to bring the job to foreground.
11. Press Ctrl+Z to suspend it.
12. Use bg to resume it.

14. Process Priority

Linux assigns a priority/scheduling value to processes. The manual introduces nice and renice.

nice: launches a process with a user-defined scheduling nice value.

renice: modifies the scheduling nice value of a running process.

Manual values: nice range = -20 to +19. -20 is highest priority, 0 is default, +19 is lowest.

The manual gives: PR = 20 + NI for the normal user-process mapping discussed there.

```
nice -n 10 gedit &
ps -l
```

The exact displayed PR/NI fields should be read from your system.

```
renice -n 5 -p PID
```

Replace PID with the actual process ID. Permissions may restrict increasing a process's priority.

15. Activity 2 – Priority

Manual task: start gedit with priority 90, then reset its priority to 65.

Important teaching clarification: the manual discusses both PR and NI, and the numbers 90 and 65 should be interpreted in the context of the process-priority display shown in the manual. Do not confuse PR with the nice value.

13. Start gedit in background.
14. Find its PID using ps.
15. Inspect its PR/NI values.
16. Use the appropriate priority/nice operation required by the lab environment.
17. Use ps again to verify the change.

16. Other Useful Process Commands

```
vmstat 5
```

Displays a snapshot/continuous view of memory, swap and I/O/resource information, updating every 5 seconds.

```
xload
```

Displays system load over time.

```
tload
```

Similar to xload but displays load information in the terminal.

17. C++ on Linux – What is Compilation?

C++ source code is written in a high-level language. A compiler translates it into code that the computer can execute. The Linux compiler used in this lab is GNU C++ (g++).

The manual explains that g++ compiles C++ source files and can be used from the Linux command line.

18. Check / Install g++

```
g++ --version
```

Purpose: checks whether g++ is installed and shows its version.

```
sudo apt-get update
```

Purpose: updates package information.

```
sudo apt install g++
```

Purpose: installs the GNU C++ compiler when needed.

19. Compile and Run a C++ Program

Step 1 – create a source file, for example hello.cpp.

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World!";
    return 0;
}
```

Step 2 – compile:

```
g++ hello.cpp
```

If no -o option is supplied, the manual explains that the executable is commonly named a.out.

```
./a.out
```

Step 3 – compile with your own executable name:

```
g++ hello.cpp -o hello
```

Step 4 – execute:

```
./hello
```

./ means “run the executable from the current directory.”

20. Complete C++ Workflow for Students

18. Open Terminal.
19. Go to the directory containing the source file.
20. Create/write the .cpp file.
21. Compile using g++.
22. Check for compilation errors.
23. Run the executable using ./program-name.

24. Correct any errors and compile again.

21. Command-Line Arguments

The manual introduces command-line arguments so the user can pass values while executing a program.

```
int main(int argc, char *argv[])
```

Term	Meaning	Example
argc	Argument count	./avg 10 20 30 → argc = 4
argv	Argument values	argv[0] = ./avg
argv[1]	First user argument	10
argv[2]	Second user argument	20
argv[3]	Third user argument	30

Why argc = 4? Because argv[0] is also counted: program name + 3 user values = 4.

```
./avg 10 20 30
```

Arguments are separated by spaces. If one argument contains spaces, use quotation marks.

22. Command-Line Average Example

The manual describes an average example in which values are passed to the executable from the command line.

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
{
    double sum = 0;

    for (int i = 1; i < argc; i++)
        sum += atof(argv[i]);

    if (argc > 1)
        cout << "Average = " << sum / (argc - 1) << endl;

    return 0;
}
```

```
g++ avg.cpp -o avg
./avg 10 20 30
```

The program starts from argv[1] because argv[0] is the executable name.

23. Activity 3 – Maximum and Minimum

The manual asks for a C++ program to find the maximum and minimum value in an array.

```
#include<iostream>
using namespace std;
```

```
int main ()
{
    int arr[10], n, i, max, min;

    cout << "Enter the size of the array : ";
    cin >> n;

    cout << "Enter the elements of the array : ";
    for (i = 0; i < n; i++)
        cin >> arr[i];

    max = arr[0];

    for (i = 0; i < n; i++)
    {
        if (max < arr[i])
            max = arr[i];
    }

    min = arr[0];

    for (i = 0; i < n; i++)
    {
        if (min > arr[i])
            min = arr[i];
    }

    cout << "Largest element : " << max;
    cout << "Smallest element : " << min;

    return 0;
}
```

24. Line-by-Line Easy Explanation of Max/Min Program

Code/Statement	Easy Meaning
int arr[10], n, i, max, min;	Creates variables. arr stores numbers; n is size; i is loop counter; max/min store answers.
cin >> n;	Reads the number of elements.
for (...) cin >> arr[i];	Reads each array element.
max = arr[0];	Initially assumes first element is maximum.
if (max < arr[i]) max = arr[i];	If current value is bigger, update max.
min = arr[0];	Initially assumes first element is minimum.
if (min > arr[i]) min = arr[i];	If current value is smaller, update min.
cout ...	Prints the final maximum and minimum.

25. Example Run of Max/Min

```
Enter the size of the array : 5
Enter the elements of the array : 10 4 25 7 2
Largest element : 25
Smallest element : 2
```

Teaching explanation: scan the array once for the largest value and once for the smallest value.

26. Graded Task 1 – Command-Line Array Input

Manual requirement: change Activity 3 so it accepts input at the command line.

Concept: instead of cin for every array value, use argc/argv to receive values when executing the program.

```
g++ maxmin.cpp -o maxmin
./maxmin 10 4 25 7 2
```

Here argv[1] ... argv[5] contain the five values.

27. Graded Task 2 – Palindrome

Manual requirement: accept a number and determine whether it is a palindrome.

Easy idea: reverse the number and compare the reverse with the original.

```
#include <iostream>
using namespace std;

int main()
{
    int n, original, reverse = 0, digit;
    cout << "Enter number: ";
    cin >> n;

    original = n;

    while (n > 0)
    {
        digit = n % 10;
        reverse = reverse * 10 + digit;
        n = n / 10;
    }

    if (original == reverse)
        cout << "Palindrome";
    else
        cout << "Not Palindrome";

    return 0;
}
```

```
Example: 121 → reverse = 121 → Palindrome
Example: 123 → reverse = 321 → Not Palindrome
```

28. Graded Task 3 – 1000 Random Numbers and Prime Count

Manual requirement: create an array of size 1000, fill it with random numbers between 2 and 100, and find how many are prime.

Easy algorithm:

25. Create int arr[1000].
26. Generate random values from 2 through 100.

27. Check each number for primality.
28. If prime, increase count.
29. Print the final count.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

bool isPrime(int n)
{
    if (n < 2) return false;

    for (int i = 2; i * i <= n; i++)
        if (n % i == 0)
            return false;

    return true;
}

int main()
{
    int arr[1000];
    int count = 0;

    srand(time(0));

    for (int i = 0; i < 1000; i++)
        arr[i] = 2 + rand() % 99;

    for (int i = 0; i < 1000; i++)
        if (isPrime(arr[i]))
            count++;

    cout << "Number of prime values = " << count << endl;
    return 0;
}
```

Note: the exact count changes between runs because the numbers are random.

29. Full Practical Demonstration – Recommended Sequence

30. Open Ubuntu/Linux Terminal.
31. Check g++ using g++ --version.
32. Run ps and identify PID.
33. Run ps -A and discuss system-wide processes.
34. Run ps -F and explain detailed information.
35. Run ps -p 1 and explain PID 1.
36. Run top -d 0.5 and quit with q.
37. Run pstree and explain process hierarchy.
38. Start gedit in background using gedit &.
39. Use jobs, fg, Ctrl+Z and bg.
40. Demonstrate process priority concepts using nice/renice and verify with ps.
41. Create hello.cpp.

42. Compile using g++ hello.cpp -o hello.
43. Execute using ./hello.
44. Explain argc/argv using ./avg 10 20 30.
45. Run the max/min program.
46. Discuss the three graded tasks.

30. Common Student Mistakes

- Writing g++ hello.cpp but forgetting to execute the output file.
- Forgetting ./ before a local executable, e.g. using hello instead of ./hello.
- Thinking a program and a process are exactly the same thing.
- Confusing PID with priority.
- Confusing PR with NI/nice value.
- Using argv[0] as the first user number; argv[0] is normally the program name.
- Forgetting that argc includes argv[0].
- Using Ctrl+Z when the intention is to terminate a process; Ctrl+Z suspends it.
- Forgetting to press q to leave top.
- Trying to increase priority without sufficient permissions.

31. Viva Questions with Easy Answers

Question	Answer
What is a process?	A running instance of a program.
What is PID?	Process ID; the number used to identify a process.
What command displays processes?	ps.
ps vs top?	ps gives a snapshot; top gives a dynamic/live view.
What is pstree?	A tree view of processes and their relationships.
What does & mean?	Run the command in the background.
What does jobs do?	Shows jobs controlled by the current shell.
What does fg do?	Brings a job to the foreground.
What does bg do?	Resumes a suspended job in the background.
What does Ctrl+Z do?	Suspends the foreground job.
What does Ctrl+C do?	Interrupts the foreground command.
What is nice?	Starts a process with a selected nice/scheduling value.
What is renice?	Changes the nice value of an existing process.
Nice value range?	-20 to +19 in the manual; -20 highest, +19 lowest.
What is g++?	GNU C++ compiler.
How compile?	g++ file.cpp -o program
How run?	./program
What is argc?	Number of command-line arguments.
What is argv?	Array containing command-line argument strings.
What is argv[0]?	Normally the program name/path.
What is a palindrome?	A value that reads the same forward and backward.
What is a prime number?	A number greater than 1 with no positive divisors other than 1 and itself.

32. Quick Command Sheet

- ps → current shell processes
- ps -A / ps -e → all active processes
- ps -x → current user's processes

- ps -F → full-format listing
- ps -u USER → user's processes
- ps -p PID → selected PID
- ps -r → running processes
- ps -L → threads
- top → live process view
- top -d 0.5 → live view with 0.5-second delay
- top -u USER → live view for a user
- pstree → process tree
- gedit & → background process
- jobs → shell jobs
- fg → foreground
- bg → resume in background
- Ctrl+Z → suspend
- Ctrl+C → interrupt
- nice → start with a nice value
- renice → change running process nice value
- g++ --version → compiler version
- g++ hello.cpp → compile; commonly produces a.out
- ./a.out → run default executable
- g++ hello.cpp -o hello → named executable
- ./hello → run named executable

33. Final Lecture Summary

The main idea of Lab 5 is divided into two parts:

- PART A – PROCESS MANAGEMENT: understand processes, PIDs, process states, ps, top, pstree, foreground/background jobs, and priority using nice/renice.
- PART B – C++ IN LINUX: write a .cpp file, compile it with g++, execute it with ./program, and understand command-line arguments using argc/argv.
- Students should be able to perform the solved activities and understand the logic of the graded tasks.

34. Instructor Reminder

This document preserves the organization and requirements of Lab No. 05 in the provided CSC322 manual. Additional code examples and teaching explanations are clearly intended to make the manual easier to deliver in class. Where a system-specific behavior can vary, students should verify the result on their Linux installation.